

Day 6 Assignment — Problem Statement

Build `mini-store.js`

Build a small state-management library in plain Node, from scratch, with no external libraries. It should behave like a stripped-down Redux, and it must use all five Day 6 concepts together rather than in isolation.

Part 1 — Immutability

Write a recursive freeze utility that locks an object and everything nested inside it, and survives nulls, functions, and circular references. Then write a pure nested-update helper: given an object, a path into it, and an updater function, it returns a new object with only that one value changed. Branches you didn't touch must come back as the **same reference** — that's the hard requirement. Deep-cloning the whole tree is not allowed.

Part 2 — Composition

Implement left-to-right and right-to-left composition helpers, then prove in code that one is the reverse of the other. Add a pass-through helper for side effects like logging, so a debug statement in the middle of a chain doesn't swallow the value. Add an async version that works when the chain mixes synchronous and asynchronous steps. Finally, make the chain accept multiple arguments on its first step.

Part 3 — Module + Singleton

Build a store factory using the module pattern. The state must be genuinely private — reachable only through a getter, invisible as a property. The getter should hand back a frozen snapshot in dev mode so anyone trying to mutate it fails loudly rather than silently. Make the store a singleton: one shared instance, direct construction throws, and there's an explicit reset hatch for tests. Crucially, freeze the state **before** passing it to the reducer, so an impure reducer crashes instead of quietly corrupting the app. Verify with two reducers — one mutating, one returning a new object. Exactly one should survive.

Part 4 — Pub-Sub

Add subscription to the store, built to survive real-world abuse. Subscribing returns an unsubscribe function. A subscriber that unsubscribes itself mid-notification must not cause the next subscriber to be skipped. A subscriber that throws must not stop the rest. Support one-shot subscribers, and a wildcard subscriber that receives every event with the event name attached. Handle re-entrancy: if a subscriber dispatches again, queue it and drain after the current pass instead of recursing. Then write the subscriber pair that would infinite-loop under a naive implementation, and show yours doesn't.

Part 5 — Factory + Decorator

Write a factory that produces action creators from a type string. Then write decorators that wrap dispatch: logging, timing, validation, async retry with backoff, and memoization keyed by a custom function rather than a blind stringify. Every decorator must preserve the wrapped function's name and arity — most people skip this, and it's why stack traces go blank in real middleware stacks. Assemble the middleware chain using your own composition helper from Part 2, then demonstrate that swapping the order of two decorators changes the output, and explain why.

Final assembly

Put a small order-tracking app on top: a reducer handling add, remove, and discount actions; three independent subscribers that each render a different part of the UI; and a fully decorated dispatch. Write assertions proving the state is private, snapshots are frozen, unsubscribe actually works, every dispatch produces a new root object, untouched branches are shared by reference, and the decorated dispatch still reports its original name.

Stretch

Add time-travel by keeping every snapshot, and explain in a comment why pure reducers make this nearly free.

Make deep-freezing a no-op in production and measure the difference over 100k dispatches.

Rewrite the whole thing as an ES module and answer whether the module system makes your Singleton class redundant.